PRACTICAL NO.5

Aim:- Using a small dataset (e.g., weather or spam dataset), implement a Naive Bayes classifier using scikit-learn.

Code:-

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

#1. Create a tiny dataset(Text, and Label: 1 for spam, 0 for ham)
corpus=[
    "Get rich quick! Click here now!", #Spam
    "Hey, are we still meeting for lunch today?", #Ham
    "Free cash prize! Claim your reward.", #Spam
    "Can you send me the project report?", #Ham
    "Urgent: Account suspended. Verification required.", #Spam
    "Don't forget to buy milk on your way home.", #Ham
    ]
labels=[1,0,1,0,1,0]

#2. Convert text into numerical vectors(Bag of words)
vectorizer=CountVectorizer()
X=vectorizer.fit_transform(corpus)
y=labels

#3. Train the Naive Bayes model(MultinomialNB is great for text)
model= MultinomialNB()
model.fit(X,y)

#4. Test the model with brand new emails
test_emails=[
    "Hey buddy, let's grab a coffee later",
    "CONGRATULATIONS! You won a free vacation! Click now.",
    ]

#Convert test text using the same vectorizer
X_test=vectorizer.transform(test_emails)
predictions=model.predict(X_test)
```

```
#5. Output the results
for email,pred in zip(test_emails,predictions):
    status="SPAM 🚨" if pred==1 else "HAM(Safe)✔"
    print(f"Email:'{email}'--> Predicted as: {status}")
```

Output:-

```
Email:'Hey buddy, let's grab a coffee later'--> Predicted as: HAM(Safe)✔
Email:'CONGRATULATIONS! You won a free vacation! Click now.'--> Predicted as: SPAM
🚨
```

PRACTICAL NO.6

Aim:- Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:-

```
import pgmpy
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model=DiscreteBayesianNetwork([('Burglary','Alarm'),('Earthquake','Alarm')])
cpd_burglary=TabularCPD(variable='Burglary',variable_card=2,values=[[0.99],[0.01]])

#Prior Probability of Earthquake: P(Earthquake)
cpd_earthquake=TabularCPD(variable='Earthquake',variable_card=2,values=[[0.998],[0.002]
])

#B=0,E=0),(B=0,E=1),(B=1,E=0),(B=1,E=1)
cpd_alarm=TabularCPD(
   variable='Alarm',
   variable_card=2,
   values=[
      [0.999,0.71,0.06,0.05], #P(Alarm=False)
      [0.001,0.29,0.94,0.95] #P(Alarm=True)
      ],
   evidence=['Burglary','Earthquake'],
   evidence_card=[2,2]
)

#3. Add CPDs to the model and validate
model.add_cpds(cpd_burglary,cpd_earthquake,cpd_alarm)
assert model.check_model(),"Model defination is invalid!"

#4. Perform Exact inference using variable elimination
infer=VariableElimination(model)

#Compute posterior probability: P(Burglary | Alarm = True)
posterior=infer.query(variables=['Burglary'],evidence={'Alarm':1})
```

```
#Display the output
print("Posterior Probability distribution P(Burglary| Alarm = True):")
print(posterior)
```

Output:-

```
Posterior Probability distribution P(Burglary| Alarm = True):
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
| Burglary(0) |          0.1425 |
+-------------+-----------------+
| Burglary(1) |          0.8575 |
+-------------+-----------------+
```